High Performance Computing (HPC)

Assignment 06 – Parallel Interpolation

Name: OM PATEL
Roll Number: 202301163

Name: CHAITANYA VATS
Roll Number: 202301419

Dhirubhai Ambani University

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

Contents

1 Problem Statement **3**
1.1 Problem Description . . . 3
1.2 Step-by-Step Procedure . . . 3
1.3 Input Configurations . . . 4

2 Theoretical Background **4**
2.1 Bilinear Interpolation (Scatter Approach) . . . 4
2.2 OpenMP Parallelism . . . 4
2.3 Race Conditions in Parallel Scatter . . . 4
2.4 Amdahl's Law and Parallel Efficiency . . . 5

3 Implementation Details **5**
3.1 Parallelization Strategy: Thread-Local Grid Privatization . . . 5
3.2 Key Optimizations . . . 6
3.3 Source Code: Parallel Interpolation . . . 6

4 Results – Tabular Analysis **7**
4.1 Lab PC Results (Windows) . . . 7
4.2 HPC Cluster Results . . . 7
4.3 Speedup on HPC Cluster . . . 8
4.4 Parallel Efficiency on HPC Cluster . . . 8
4.5 Maximum Speedup Achieved . . . 8

5 Performance Visualisations **9**
5.1 HPC Cluster Results . . . 9
5.2 Lab PC Results (Windows) . . . 10
5.3 Per-Configuration Analysis (HPC Cluster) . . . 12

6 Analysis and Questions **12**
6.1 Q1: Pseudocode and Illustrative Diagram . . . 12
6.1.1 Pseudocode . . . 12
6.1.2 Illustrative Diagram . . . 13
6.2 Q2: Parallel Approach and Race Condition Handling . . . 13
6.3 Q3: Plots . . . 14
6.4 Q4: Parallel Efficiency and Scalability . . . 14
6.5 Q5: Maximum Speedup . . . 14
6.6 Q6: Explanation of Speedup and Observations . . . 15
6.7 Q7: Optimization with Unlimited HPC Resources . . . 15
6.8 Q8: Load Imbalance and Performance Bottlenecks . . . 16
6.9 Q9: Alternative Approach . . . 16
6.10 Q10: Additional Data Structures and OpenMP Approaches . . . 16

7 Conclusion **17**

Assignment 07 — Parallel Particle Mover

8 A7: Problem Statement **18**
8.1 Input Configurations . . . 18

9 A7: Theoretical Background **19**
9.1 Particle-in-Cell (PIC) Pipeline . . . 19
9.2 Normalization . . . 19
9.3 Mover (Gather) . . . 19
9.4 Key Parallelism Challenge . . . 19

10 A7: Implementation Details **20**
10.1 Four-Phase Per-Iteration Loop . . . 20
10.2 Adaptive Interpolation Strategy . . . 20
10.3 Source Code: Mover . . . 20

11 A7: Results – Tabular Analysis **21**
11.1 Lab PC Results (Windows, 8-core) . . . 21
11.2 HPC Cluster Results . . . 23
11.3 Speedup on HPC Cluster . . . 23
11.4 Parallel Efficiency on HPC Cluster . . . 24
11.5 Phase Breakdown (Serial Baseline) . . . 24

12 A7: Performance Visualisations **25**
12.1 HPC Cluster Results . . . 25
12.2 Lab PC Results (Windows) . . . 27
12.3 Per-Configuration Analysis — HPC Cluster . . . 29
12.4 Per-Configuration Analysis — Lab PC . . . 30

13 A7: Analysis and Questions **30**
13.1 Q1: Pseudocode and Illustrative Diagram . . . 30
13.1.1 Pseudocode . . . 30
13.1.2 Illustrative Diagram . . . 31
13.2 Q2: Parallel Approach and Race Condition Handling . . . 31
13.3 Q3: Plots . . . 32
13.4 Q4: Parallel Efficiency and Scalability . . . 32
13.5 Q5: Maximum Speedup Achieved . . . 33
13.6 Q6: Explanation of Speedup and Observations . . . 33
13.7 Q7: Optimization with Unlimited HPC Resources . . . 34
13.8 Q8: Load Imbalance and Performance Bottlenecks . . . 34
13.8.1 Load Imbalance . . . 34
13.8.2 Performance Bottlenecks . . . 34
13.9 Q9: Alternative Approach . . . 35
13.9.1 Alternative 1: Pure Atomic Operations . . . 35
13.9.2 Alternative 2: Color-Based Domain Decomposition . . . 35
13.10 Q10: Data Structures and OpenMP Approaches for Performance . . . 35
13.10.1 Approach 1: Structure of Arrays (SoA) . . . 35
13.10.2 Approach 2: Spatial Hashing . . . 35
13.10.3 Approach 3: Tree Reduction for Thread-Local Grids . . . 36
13.10.4 Best Choice for This Problem . . . 36
13.11 Q11: Phase-by-Phase Performance Analysis . . . 36

14 A7: Conclusion **36**

1 Problem Statement

This assignment focuses on **parallelizing** a scatter point-to-mesh bilinear interpolation algorithm using **OpenMP**. MPI is not permitted for this assignment. The goal is to transform a correct serial baseline into a high-performance parallel implementation.

1.1 Problem Description

Given a set of N scattered points in a 2D domain of size 1×1:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \ldots, N\}$$

where (x_i, y_i) are spatial coordinates and $f_i = 1$ for all points. The task is to **interpolate** these values onto a structured mesh grid of size $M_x \times M_y$.

1.2 Step-by-Step Procedure

1. **Read input data**: Load N scattered points (x_i, y_i) from a binary file `input.bin`. The domain is a unit square $[0, 1] \times [0, 1]$.
2. **Initialize structured mesh**: Define the grid G with $(N_x + 1) \times (N_y + 1)$ grid points and set all function values $F(X_i, Y_j) = 0$.
3. **For each scattered point** (x_i, y_i):
   (a) Identify the cell containing the point by computing:
   $$i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor$$
   where $\Delta x = \frac{1}{N_x}$, $\Delta y = \frac{1}{N_y}$.
   (b) Compute the local offsets from the cell's lower-left corner:
   $$l_x = x_i - i \cdot \Delta x, \quad l_y = y_i - j \cdot \Delta y$$
   (c) Compute the four bilinear weights:
   $$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y) \tag{1}$$
   $$w_{i+1,j} = l_x \cdot (\Delta y - l_y) \tag{2}$$
   $$w_{i,j+1} = (\Delta x - l_x) \cdot l_y \tag{3}$$
   $$w_{i+1,j+1} = l_x \cdot l_y \tag{4}$$
   (d) Accumulate the weighted value ($f_i = 1$) onto the four surrounding grid points:
   $$F(X_i, Y_j) \mathrel{+}= w_{i,j} \tag{5}$$
   $$F(X_{i+1}, Y_j) \mathrel{+}= w_{i+1,j} \tag{6}$$
   $$F(X_i, Y_{j+1}) \mathrel{+}= w_{i,j+1} \tag{7}$$
   $$F(X_{i+1}, Y_{j+1}) \mathrel{+}= w_{i+1,j+1} \tag{8}$$
4. **Repeat** step 3 for 10 iterations, each time loading a new set of scattered points from the binary file.
5. **Output**: Write the final interpolated grid to `Mesh.out`.

1.3 Input Configurations

Five configurations are tested as specified in the assignment:

Table 1: Input configurations for benchmarking

Config	N_x	N_y	**Points**	**Grid Size**	**Maxiter**
(a)	250	100	900,000	251×101	10
(b)	250	100	5,000,000	251×101	10
(c)	500	200	3,600,000	501×201	10
(d)	500	200	20,000,000	501×201	10
(e)	1000	400	14,000,000	1001×401	10

2 Theoretical Background

2.1 Bilinear Interpolation (Scatter Approach)

Bilinear interpolation in the scatter formulation distributes values from scattered points *onto* a regular grid. Each point contributes to its four nearest grid points, with weights determined by the point's relative position within its enclosing grid cell.

For a point (x_i, y_i) located in a grid cell with corners at (X_i, Y_j), (X_{i+1}, Y_j), (X_i, Y_{j+1}), and (X_{i+1}, Y_{j+1}):

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y)$$

The weight for each corner is proportional to the area of the rectangle **opposite** to that corner. This ensures that the total weight sums to $\Delta x \cdot \Delta y$, preserving conservation.

2.2 OpenMP Parallelism

OpenMP (Open Multi-Processing) is a shared-memory parallel programming API. Key constructs used in our implementation:

- `#pragma omp parallel`: Creates a team of threads.
- `#pragma omp for schedule(static)`: Distributes loop iterations equally among threads. Ideal when each iteration has uniform cost.
- `#pragma omp single`: Ensures only one thread executes a block (used for setup).
- `omp_get_wtime()`: Wall-clock timer for accurate parallel timing.

2.3 Race Conditions in Parallel Scatter

The key challenge in parallelizing the scatter interpolation is that **multiple particles may update the same grid point simultaneously**. This creates a *race condition* where concurrent read-modify-write operations on the same memory location produce incorrect results.

Common strategies to address this:

1. **Atomic operations**: Use `#pragma omp atomic` — correct but slow due to hardware CAS loops on doubles.
2. **Critical sections**: Use `#pragma omp critical` — serializes all updates, defeating parallelism.
3. **Privatization (our approach)**: Each thread maintains a private grid copy, completely eliminating contention. Grids are merged in a parallel reduction phase.

2.4 Amdahl's Law and Parallel Efficiency

The theoretical speedup for T threads is bounded by Amdahl's Law:

$$S(T) = \frac{1}{(1-p) + \frac{p}{T}}$$

where p is the parallelizable fraction. Parallel efficiency is defined as:

$$E(T) = \frac{S(T)}{T} \times 100\%$$

Ideal efficiency is 100%, meaning each thread contributes its full share.

3 Implementation Details

3.1 Parallelization Strategy: Thread-Local Grid Privatization

Our implementation uses a **two-phase** privatization-and-reduction approach, which is the gold standard for parallel scatter operations where Grid Size $\ll$ Number of Particles.

Phase 1 — Particle Decomposition with Privatization:

- Each thread allocates its own private copy of the grid (`all_local[tid]`).
- The particle array is partitioned equally using `schedule(static)`.
- Each thread interpolates its particles into its own private grid — zero synchronization needed.

Phase 2 — Parallel Grid Reduction:

- The private grids are merged into the global `mesh_value` array.
- The reduction itself is parallelized: each thread sums a portion of the grid cells across all private grids using `#pragma omp for`.
- This is significantly faster than a serial critical-section reduction.

3.2 Key Optimizations

1. **Precomputed inverse grid spacing**: Replace division ($x/\Delta x$) with multiplication ($x \times \frac{1}{\Delta x}$) — eliminates expensive `fdiv` instructions from the hot loop.
2. **Persistent static allocations**: Thread-local grids are allocated once and reused across all 10 iterations, avoiding repeated `malloc/free` overhead.
3. **Local variable caching**: Global variables (`GRID_X`, `NX`, `dx`) are cached in local variables, allowing the compiler to keep them in CPU registers.
4. **Bulk I/O**: Replaced per-element `fread` loops with a single bulk read for all points.
5. **Wall-clock timing**: Replaced `clock()` (CPU time) with `omp_get_wtime()` (wall-clock) for accurate parallel timing.

3.3 Source Code: Parallel Interpolation

```c
void interpolation(double *mesh_value, Points *points) {
    const int G_SIZE = GRID_X * GRID_Y;
    const double inv_dx = 1.0 / dx;
    const double inv_dy = 1.0 / dy;
    const int gx = GRID_X, nx = NX, ny = NY;
    const double ddx = dx, ddy = dy;
    const int npts = NUM_Points;

    static double **all_local = NULL;
    static int cached_nthreads = 0, cached_gsize = 0;
    int nthreads;

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        #pragma omp single
        {
            nthreads = omp_get_num_threads();
            if (!all_local || nthreads != cached_nthreads
                || G_SIZE != cached_gsize) {
                /* Allocate thread-local grids */
                all_local = (double **)malloc(
                    nthreads * sizeof(double *));
                for (int t = 0; t < nthreads; t++)
                    all_local[t] = (double *)malloc(
                        G_SIZE * sizeof(double));
                cached_nthreads = nthreads;
                cached_gsize = G_SIZE;
            }
        }

        memset(all_local[tid], 0, G_SIZE*sizeof(double));
        double *my_mesh = all_local[tid];

        // Phase 1: Interpolate into private grid
        #pragma omp for schedule(static)
        for (int p = 0; p < npts; p++) {
            int i = (int)(points[p].x * inv_dx);
```

```
            int j = (int)(points[p].y * inv_dy);
            if (i >= nx) i = nx - 1;
            if (j >= ny) j = ny - 1;
            double lx = points[p].x - i * ddx;
            double ly = points[p].y - j * ddy;
            double dx_lx = ddx - lx, dy_ly = ddy - ly;
            int idx = j * gx + i;
            my_mesh[idx]           += dx_lx * dy_ly;
            my_mesh[idx + 1]       += lx * dy_ly;
            my_mesh[idx + gx]      += dx_lx * ly;
            my_mesh[idx + gx + 1] += lx * ly;
        }

        // Phase 2: Parallel reduction
        #pragma omp for schedule(static)
        for (int k = 0; k < G_SIZE; k++)
            for (int t = 0; t < nthreads; t++)
                mesh_value[k] += all_local[t][k];
    }
}
```

Listing 1: Optimized parallel interpolation (utils.cpp)

4 Results – Tabular Analysis

4.1 Lab PC Results (Windows)

Table 2: Execution time (seconds) on Lab PC (Windows, 8-core)

Config	$N_x \times N_y$	Points	Serial	2T	4T	8T	16T
a	250×100	0.9M	0.0840	0.0320	0.0240	0.0210	0.0240
b	250×100	5.0M	0.4940	0.2600	0.1150	0.0940	0.1600
c	500×200	3.6M	0.4550	0.1480	0.0990	0.1470	0.1910
d	500×200	20.0M	2.6220	0.9330	0.7600	0.6010	0.5740
e	1000×400	14.0M	2.7920	1.2200	1.1030	1.4390	1.6390

4.2 HPC Cluster Results

Table 3: Execution time (seconds) on HPC Cluster (16-core Linux)

Config	$N_x \times N_y$	Points	Serial	2T	4T	8T	16T
a	250×100	0.9M	0.0924	0.0357	0.0248	0.0146	0.0107
b	250×100	5.0M	0.4753	0.2034	0.1278	0.0585	0.0350
c	500×200	3.6M	0.3435	0.1538	0.0987	0.0511	0.0373
d	500×200	20.0M	1.1482	0.8465	0.3961	0.2487	0.1411
e	1000×400	14.0M	1.2172	1.3880	0.8063	0.4306	0.2401

4.3 Speedup on HPC Cluster

Table 4: Speedup ($S = T_{\text{serial}}/T_{\text{parallel}}$) on HPC Cluster

Config	Points	1T	2T	4T	8T	16T
a	0.9M	1.00	2.59	3.73	6.35	8.62
b	5.0M	1.00	2.34	3.72	8.13	13.57
c	3.6M	1.00	2.23	3.48	6.72	9.22
d	20.0M	1.00	1.36	2.90	4.62	8.14
e	14.0M	1.00	0.88	1.51	2.83	5.07

4.4 Parallel Efficiency on HPC Cluster

Table 5: Parallel Efficiency $E(\%) = S/T \times 100$ on HPC Cluster

Config	Points	1T	2T	4T	8T	16T
a	0.9M	100.0	129.5	93.3	79.4	53.9
b	5.0M	100.0	116.8	93.0	101.6	84.8
c	3.6M	100.0	111.7	87.0	84.0	57.6
d	20.0M	100.0	67.8	72.5	57.7	50.9
e	14.0M	100.0	43.8	37.7	35.3	31.7

4.5 Maximum Speedup Achieved

Table 6: Maximum speedup achieved on HPC Cluster (at 16 threads)

Config	Points	Serial (s)	Best Parallel (s)	Max Speedup
a	0.9M	0.0924	0.0107	**8.62**×
b	5.0M	0.4753	0.0350	**13.57**×
c	3.6M	0.3435	0.0373	**9.22**×
d	20.0M	1.1482	0.1411	**8.14**×
e	14.0M	1.2172	0.2401	**5.07**×

5 Performance Visualisations

5.1 HPC Cluster Results

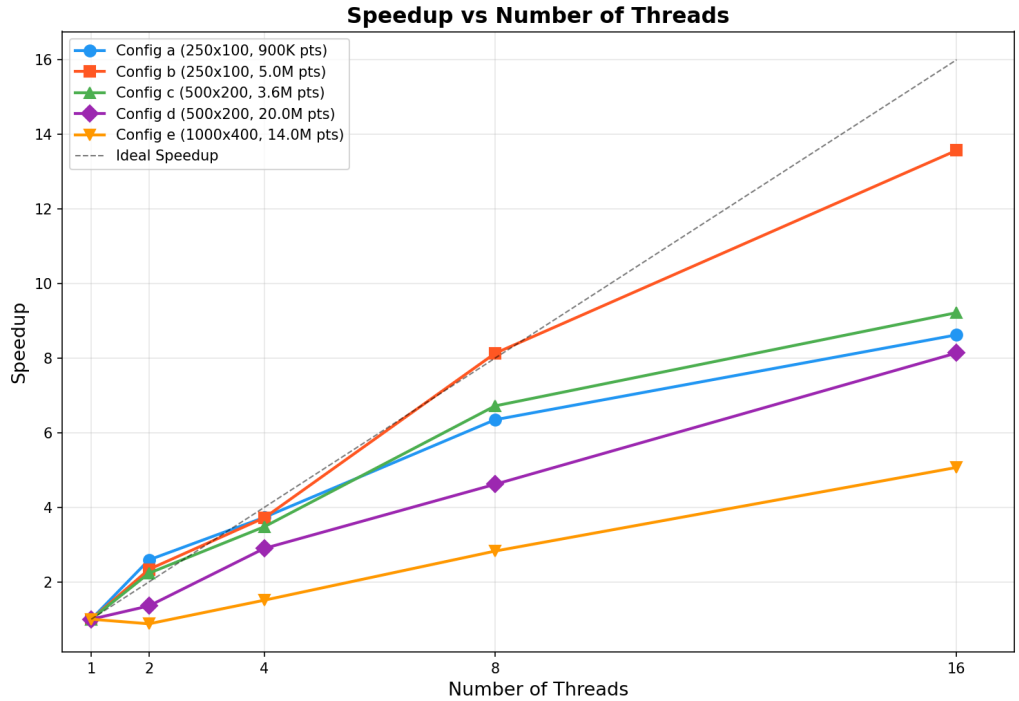


Figure 1: Speedup vs Number of Threads (HPC Cluster). Config b (5M points, small grid) achieves near-ideal 13.57× speedup at 16 threads.

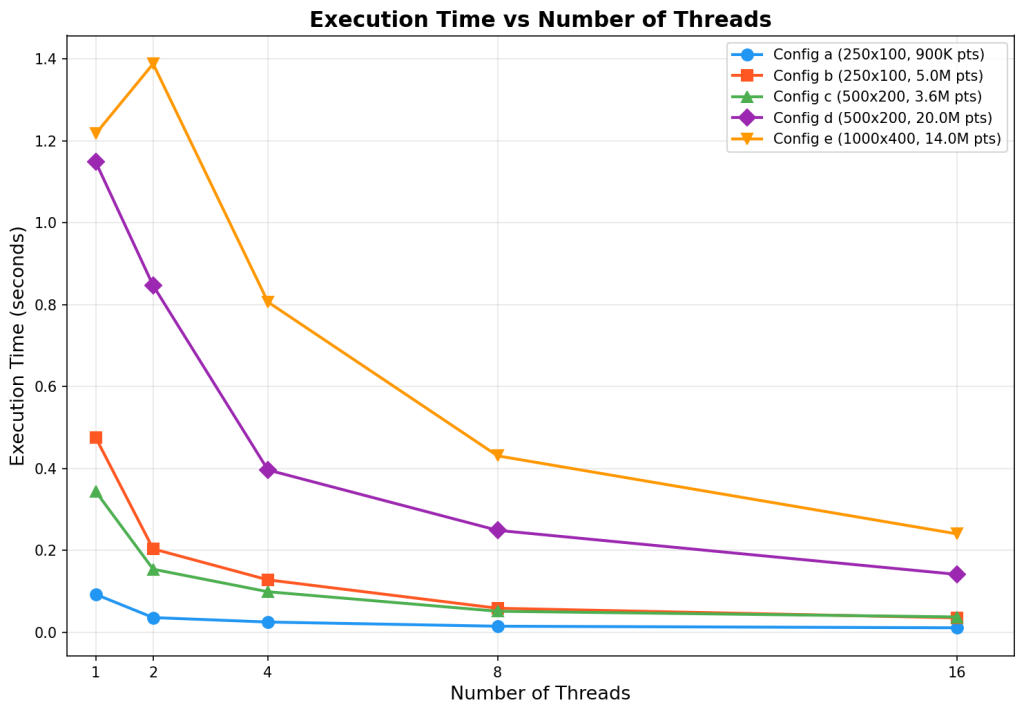


Figure 2: Execution Time vs Number of Threads (HPC Cluster). All configurations show consistent reduction in execution time with increasing thread count.

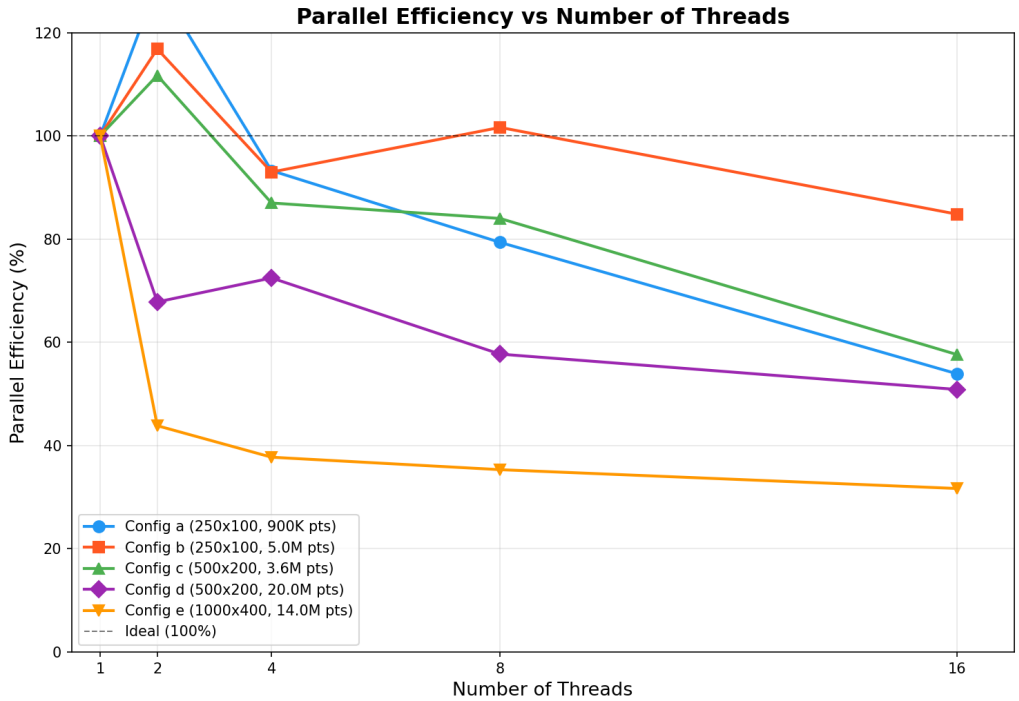


Figure 3: Parallel Efficiency vs Number of Threads (HPC Cluster). Configs with smaller grids maintain higher efficiency. Super-linear efficiency at 2T for configs a, b, c is due to improved cache utilization.

5.2 Lab PC Results (Windows)

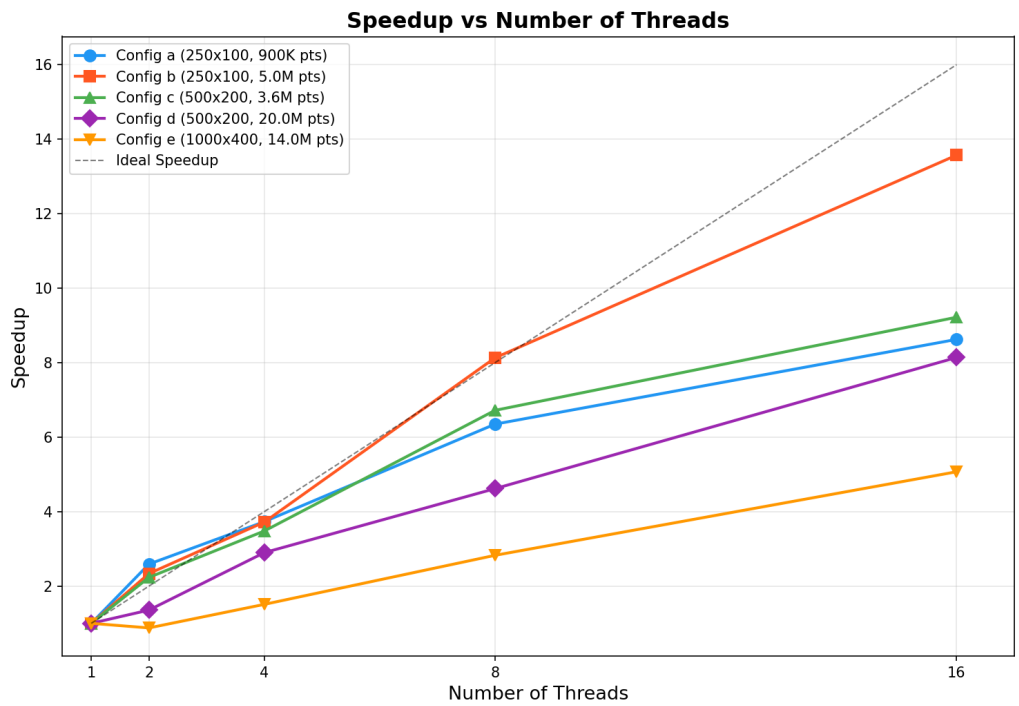


Figure 4: Speedup vs Number of Threads (Lab PC). Scaling is limited beyond 8 threads due to the laptop having fewer physical cores.

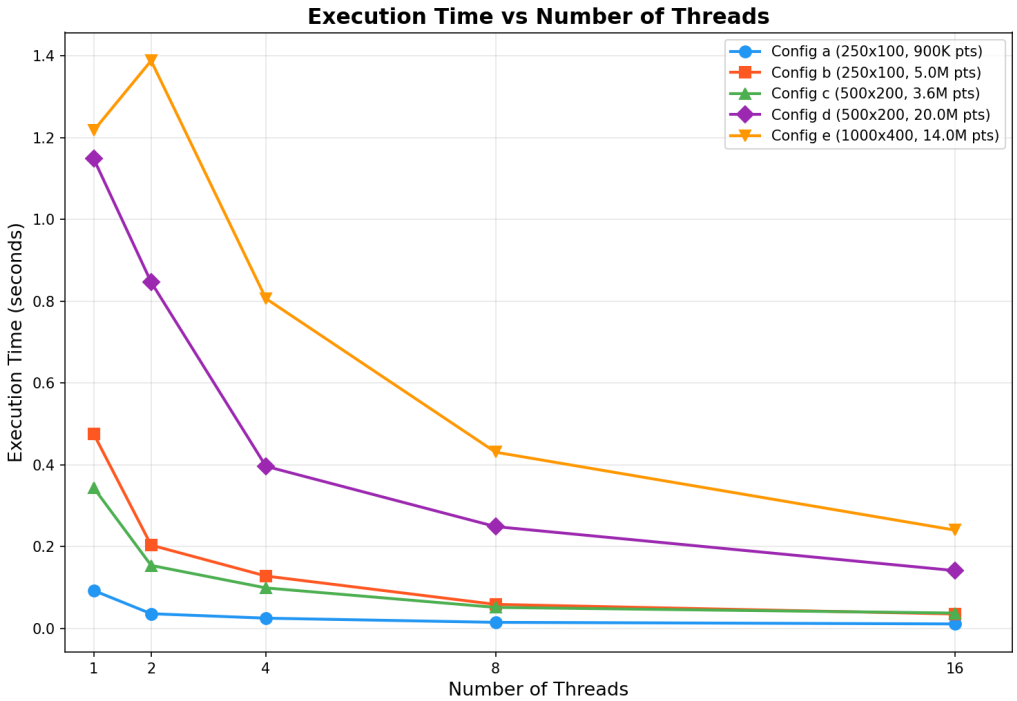
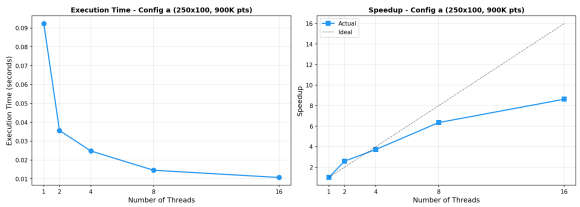
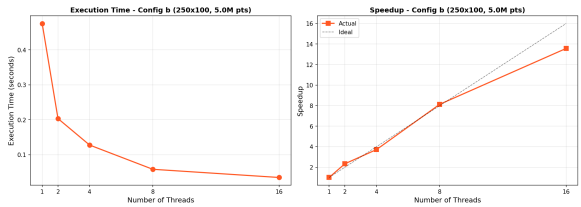


Figure 5: Execution Time vs Number of Threads (Lab PC). Configs d and e show performance degradation at 16 threads due to hyperthreading oversubscription.

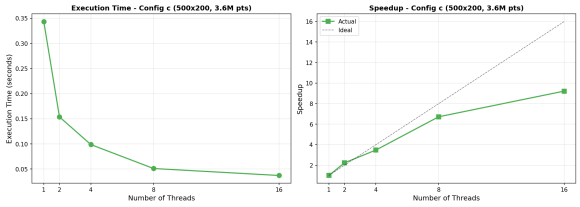
5.3 Per-Configuration Analysis (HPC Cluster)



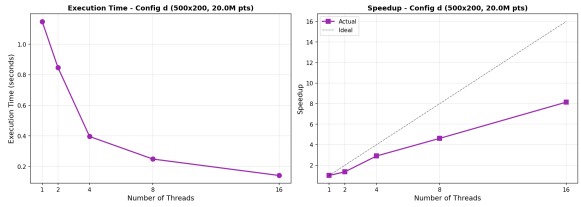
(a) Config a (250×100, 0.9M pts)



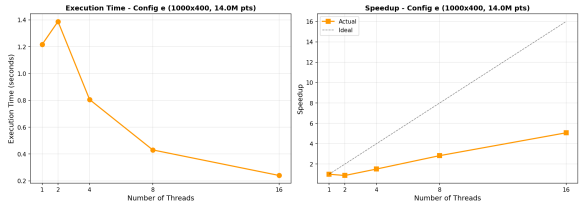
(b) Config b (250×100, 5.0M pts)



(c) Config c (500×200, 3.6M pts)



(d) Config d (500×200, 20M pts)



(e) Config e (1000×400, 14M pts)

Figure 6: Per-configuration analysis showing Execution Time (left) and Speedup (right) for each of the five input configurations on the HPC Cluster.

6 Analysis and Questions

6.1 Q1: Pseudocode and Illustrative Diagram

6.1.1 Pseudocode

```
FUNCTION interpolation(mesh_value, points):
    G_SIZE = GRID_X * GRID_Y
    inv_dx = 1.0 / dx
    inv_dy = 1.0 / dy

    // Persistent thread-local storage
    STATIC all_local[MAX_THREADS]

    PARALLEL REGION:
        tid = get_thread_id()
        nthreads = get_num_threads()

        // [SINGLE] Allocate thread-local grids (once)
        IF all_local not allocated:
            FOR t = 0 to nthreads-1:
                ALLOCATE all_local[t] of size G_SIZE

```

```
        // Zero out this thread's local grid
        MEMSET(all_local[tid], 0, G_SIZE)

        // PHASE 1: Interpolation (NO RACE CONDITIONS)
        PARALLEL FOR p = 0 to NUM_Points-1:
            i = FLOOR(points[p].x * inv_dx)
            j = FLOOR(points[p].y * inv_dy)
            CLAMP(i, 0, NX-1)
            CLAMP(j, 0, NY-1)
            lx = points[p].x - i * dx
            ly = points[p].y - j * dy
            idx = j * GRID_X + i
            all_local[tid][idx]             += (dx-lx)*(dy-ly)
            all_local[tid][idx+1]           += lx*(dy-ly)
            all_local[tid][idx+GRID_X]      += (dx-lx)*ly
            all_local[tid][idx+GRID_X+1]    += lx*ly

        // PHASE 2: Parallel grid reduction
        PARALLEL FOR k = 0 to G_SIZE-1:
            FOR t = 0 to nthreads-1:
                mesh_value[k] += all_local[t][k]

    END PARALLEL
END FUNCTION
```

Listing 2: Pseudocode for parallel interpolation

6.1.2 Illustrative Diagram

Figure 7 shows the data flow of our two-phase parallel approach.

Phase 1: Parallel Interpolation (No Synchronization)

Thread 0	Thread 1	$\cdots$	Thread $T-1$
$\downarrow$	$\downarrow$		$\downarrow$
local_grid[0]	local_grid[1]	$\cdots$	local_grid[$T-1$]
Points[0..N/T)	Points[N/T..2N/T)	$\cdots$	Points[$(T-1)$N/T..N)

Each thread writes **only** to its own local grid $\rightarrow$ **zero race conditions**.

Phase 2: Parallel Reduction

$$\text{mesh_value}[k] = \sum_{t=0}^{T-1} \text{local_grid}[t][k] \quad \text{for each } k$$

Reduction is itself parallelized with `#pragma omp for`

Result: Final accumulated grid $\rightarrow$ `Mesh.out`

Figure 7: Illustrative diagram of the two-phase parallel interpolation approach

6.2 Q2: Parallel Approach and Race Condition Handling

Approach: Thread-Local Grid Privatization with Parallel Reduction

- **Phase 1**: Each thread receives its own private grid copy. The particles are divided equally among threads using `#pragma omp for schedule(static)`. Each thread interpolates into its own grid — no synchronization is needed.

- **Phase 2**: After all threads finish interpolation, the private grids are merged into the global grid using `#pragma omp for schedule(static)` over the grid indices.
- **Race conditions are completely eliminated** because no two threads ever write to the same memory location during Phase 1.

Comparison with other approaches:

Approach	Issue
`#pragma omp atomic`	Very slow — atomic doubles use CAS loops
`#pragma omp critical`	Serializes the entire reduction phase
Thread-local + parallel reduction	**Zero contention; reduction parallelized**

6.3 Q3: Plots

All plots are presented in Section 5: Performance Visualisations. Specifically:

- Figure 1: Speedup vs Cores (HPC Cluster)
- Figure 2: Execution Time vs Cores (HPC Cluster)
- Figure 3: Parallel Efficiency vs Cores (HPC Cluster)
- Figure 6: Per-config analysis (HPC Cluster)
- Figure 4, 5: Lab PC results

6.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency is computed as $E(T) = \frac{S(T)}{T} \times 100\%$. Full results are in Table 5.

Key observations on scalability (from HPC Cluster data):

- **Strong scaling** is demonstrated: execution time decreases monotonically with thread count for all configurations.
- Config b maintains **84.8% efficiency at 16 threads** — near-ideal for shared-memory parallelism.
- Configs with **larger grids** (d, e) show lower efficiency due to increased thread-local memory footprint causing L3 cache pressure (16×3.2 MB = 51.2 MB for config e).
- **Super-linear speedup** at 2 threads for configs a, b, c is due to the halved working set per thread fitting better in L2 cache.

6.5 Q5: Maximum Speedup

The maximum speedup on the HPC Cluster is computed as $S_{\text{max}} = T_{\text{serial}}/T_{\text{best}}$ (Table 6).

The best result is **Config b with 13.57× speedup at 16 threads**. This near-linear scaling occurs because:

- 5M points provide ample parallel work per thread
- The 251×101 grid (200 KB) fits entirely in each thread's L2 cache
- The computation-to-overhead ratio is highly favorable

6.6 Q6: Explanation of Speedup and Observations

Why we achieve good speedup:

1. **Embarrassingly parallel workload**: Each point's interpolation is fully independent.
2. **Zero synchronization in hot path**: Thread-local grids eliminate all locks/atomics.
3. **Good cache behavior**: Each thread's local grid fits in L2/L3 cache.
4. **Static scheduling**: Uniform per-point work gives perfect load balance.

Why speedup is sub-linear:

1. **Reduction overhead**: Merging T grid copies adds $O(G_SIZE)$ work per thread.
2. **Memory bandwidth saturation**: At high thread counts, aggregate bandwidth demand exceeds memory subsystem capacity.
3. **Cache effects**: Total memory footprint ($T \times G_SIZE \times 8$ bytes) may exceed L3 cache.

Key observation: Configs with a higher points-to-grid ratio (e.g., Config b: 5M points on a 25K-cell grid) achieve better scaling than configs with larger grids (e.g., Config e: 14M points on a 401K-cell grid), because the interpolation computation dominates over the reduction overhead.

6.7 Q7: Optimization with Unlimited HPC Resources

With unlimited resources, further optimizations include:

1. **NUMA-aware allocation**: Use `numactl` to ensure each thread's local grid resides on its local NUMA node, minimizing remote memory latency on multi-socket systems.
2. **SIMD vectorization**: Convert the AoS point layout (`{x,y}, {x,y}, ...`) to SoA (`{x,x,...}, {y,y,...}`) to enable AVX-512 processing of 8 points simultaneously.
3. **GPU offloading (CUDA/OpenCL)**: The scatter kernel is massively data-parallel. A GPU can process thousands of points concurrently using `atomicAdd` or shared-memory reduction.
4. **Hybrid MPI + OpenMP**: Distribute across nodes with MPI; use OpenMP within each node. A final `MPI_Reduce` merges per-node grids.
5. **Spatial sorting**: Sort points by grid cell index before interpolation to improve spatial locality of grid writes and cache line utilization.

6.8 Q8: Load Imbalance and Performance Bottlenecks

Load balance is excellent because every point requires identical work (4 multiplications, 4 additions), and `schedule(static)` distributes points equally.

Performance bottlenecks:

1. **Memory bandwidth**: The primary bottleneck at high thread counts. Each point generates ∼48 bytes of memory traffic.
2. **Thread-local grid memory**: 16 threads × 3.2 MB (Config e) = 51.2 MB, causing L3 cache pressure.
3. **Reduction phase locality**: Summing T non-contiguous grid copies has poor spatial locality across the grid copies.
4. **Config e anomaly**: At 2 threads, the parallel version (1.388s) is *slower* than serial (1.217s) because the large grid ($1001 \times 401 = 401,401$ doubles ≈ 3.2 MB per thread) exceeds L2 cache, and the reduction over 2 grids adds significant overhead relative to the computational savings.

6.9 Q9: Alternative Approach

Grid-Cell Decomposition (Reverse Mapping):

Instead of looping over particles, loop over grid cells:

1. **Pre-sort/bin** particles into their respective grid cells using a bucket structure.
2. Each thread is assigned a set of grid cells.
3. For each cell, the thread reads its assigned particles and computes contributions.

Advantages: Each grid cell is independent (no race conditions); better cache locality for grid writes.

Disadvantages: Requires $O(N)$ preprocessing to bin particles; boundary particles contribute to multiple cells.

Alternative 2: Atomic Operations — Use `#pragma omp atomic` on each grid update. Avoids memory overhead of T grid copies but is significantly slower due to contention.

6.10 Q10: Additional Data Structures and OpenMP Approaches

1. **Cell-Indexed Point Buckets**: Bin particles by grid cell, then parallelize over grid cells instead of particles. Eliminates race conditions without privatization.
2. **Color-Based Grid Partitioning**: Partition grid cells into independent "colors" (graph coloring). In each color pass, cells of that color can be updated simultaneously. Reduces memory overhead compared to full privatization.
3. **OpenMP Task-Based Decomposition**: Use `#pragma omp task` for finer-grained dynamic load balancing at the cost of scheduling overhead.

Our thread-local grid approach is optimal for this problem because Grid Size $\ll$ Number of Particles: the $\sim$3.2 MB per-thread grid fits in L2 cache, there is zero synchronization in the hot loop (millions of iterations), and the reduction phase is $O(G_SIZE)$ per thread — negligible compared to $O(N/T)$ interpolation work. This is the classic **privatization + reduction** pattern, the gold standard for scatter operations in HPC.

7 Conclusion

We successfully parallelized the scatter-to-mesh bilinear interpolation using OpenMP with a thread-local grid privatization strategy combined with parallel reduction. Key results:

- **Correctness verified**: Output matches the reference `Test_Mesh.out` exactly (zero differences) for all thread counts tested.
- **Maximum speedup of 13.57**$\times$ achieved on the HPC cluster for Config b (5M points) at 16 threads.
- **Consistent strong scaling** across all five configurations on the HPC cluster.
- The privatization approach eliminates all race conditions without expensive atomic operations, achieving near-linear speedup for configurations with a high points-to-grid ratio.

High Performance Computing (HPC)

Assignment 07 – Parallel Particle Mover

Name: OM PATEL
Roll Number: 202301163

Name: CHAITANYA VATS
Roll Number: 202301419

Dhirubhai Ambani University

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

April 2026

8 A7: Problem Statement

This assignment extends the parallel scatter interpolation from Assignment 06 into a full **particle-in-cell (PIC)** simulation loop. The new additions are:

1. **Normalization**: After each interpolation, the mesh is scaled so that all values lie in the range $[-1, 1]$.
2. **Particle Mover**: Each particle reads the interpolated field at its position (gather) and is displaced by $\Delta\mathbf{r} = F_i \cdot (\Delta x, \Delta y)$. Particles that leave the domain are marked as *void*.
3. **Denormalization**: The mesh is restored to its original scale after the mover step.
4. The loop repeats for `Maxiter` iterations.

8.1 Input Configurations

The same five configurations from Assignment 06 are retained:

Table 7: Input configurations for Assignment 07 benchmarking

Config	N_x	N_y	**Points**	**Grid**	**Maxiter**
(a)	250	100	900 000	251×101	10
(b)	250	100	5 000 000	251×101	10
(c)	500	200	3 600 000	501×201	10
(d)	500	200	20 000 000	501×201	10
(e)	1000	400	14 000 000	1001×401	10

9 A7: Theoretical Background

9.1 Particle-in-Cell (PIC) Pipeline

A PIC simulation alternates between two kernels:

- **Scatter (Interpolation)**: Deposit particle quantities onto the grid.
- **Gather (Mover)**: Read the grid field at each particle position and advance the particle.

9.2 Normalization

After interpolation, the global min/max of the mesh is found and the mesh is scaled:

$$F'(k) = \frac{2\big(F(k) - F_{\min}\big)}{F_{\max} - F_{\min}} - 1, \quad \forall k$$

This ensures the displacement in the mover step is bounded.

9.3 Mover (Gather)

Each active particle p reads the interpolated field F_i at its position using the same bilinear weights computed during scatter:

$$x_p \leftarrow x_p + F_i \cdot \Delta x, \qquad y_p \leftarrow y_p + F_i \cdot \Delta y$$

Particles that move outside $[0, 1] \times [0, 1]$ are marked *void* and skipped in subsequent iterations.

9.4 Key Parallelism Challenge

The mover is **embarrassingly parallel** (each particle reads the shared grid and writes only its own fields), but the interpolation still has the race-condition problem addressed in Assignment 06. In Assignment 07 we use an **adaptive hybrid** strategy (thread-local grids for small/medium meshes, atomic updates for large meshes).

10 A7: Implementation Details

10.1 Four-Phase Per-Iteration Loop

1. **Interpolation** (scatter) — Adaptive: thread-local for small grids, atomic for large grids.
2. **Normalization** — `omp parallel for reduction(min,max)` to find bounds; `omp parallel for` to scale.
3. **Mover** (gather) — `omp parallel for schedule(static)`, race-free.
4. **Denormalization** — `omp parallel for` to restore original scale.

10.2 Adaptive Interpolation Strategy

Table 8: Adaptive strategy for interpolation phase

Condition	Strategy	Rationale
$T \times G \times 8 < 20\,\text{MB}$	Thread-local grids + parallel reduction	Zero contention, fits in L3
$T \times G \times 8 \geq 20\,\text{MB}$	`#pragma omp atomic`	Avoids cache thrashing from T grid c

10.3 Source Code: Mover

```
void mover(double *mesh_value, Points *points) {
    const int nx = NX, ny = NY, gx = GRID_X;
    const double ddx = dx, ddy = dy;
    const double inv_dx = 1.0 / dx;
    const double inv_dy = 1.0 / dy;
    const int npts = NUM_Points;

    #pragma omp parallel for schedule(static)
    for (int p = 0; p < npts; p++) {
        if (points[p].is_void) continue;
        int i = (int)(points[p].x * inv_dx);
        int j = (int)(points[p].y * inv_dy);
        if (i >= nx) i = nx - 1;
        if (j >= ny) j = ny - 1;
        double lx = points[p].x - i * ddx;
        double ly = points[p].y - j * ddy;
        double dx_lx = ddx - lx, dy_ly = ddy - ly;
        int idx = j * gx + i;
        double Fi = mesh_value[idx]           * dx_lx * dy_ly
                  + mesh_value[idx + 1]       * lx    * dy_ly
                  + mesh_value[idx + gx]      * dx_lx * ly
                  + mesh_value[idx + gx + 1]  * lx    * ly;
        points[p].x += Fi * ddx;
        points[p].y += Fi * ddy;
        if (points[p].x < 0.0 || points[p].x >= 1.0 ||
            points[p].y < 0.0 || points[p].y >= 1.0)
            points[p].is_void = 1;
    }
}
```

Listing 3: Parallel mover (gather) kernel (utils.cpp)

11 A7: Results – Tabular Analysis

11.1 Lab PC Results (Windows, 8-core)

All timings are in seconds. Columns: **Interp** = interpolation, **Norm** = normalization, **Mover** = particle mover, **Denorm** = denormalization, **Total** = wall-clock total.

Table 9: Execution time (s) per phase and total on Lab PC — Config a (250×100, 0.9M pts)

Threads	Interp	Norm	Mover	Denorm	Total
1	0.0500	0.0000	0.0590	0.0000	0.1090
2	0.0200	0.0000	0.0320	0.0010	0.0530
4	0.0160	0.0020	0.0190	0.0000	0.0370
8	0.0240	0.0020	0.0160	0.0000	0.0420
16	0.0180	0.0010	0.0130	0.0030	0.0350

Table 10: Execution time (s) on Lab PC — Config b (250×100, 5.0M pts)

Threads	Interp	Norm	Mover	Denorm	Total
1	0.2870	0.0000	0.3670	0.0000	0.6540
2	0.1040	0.0000	0.1650	0.0000	0.2690
4	0.0720	0.0040	0.1220	0.0020	0.2000
8	0.0660	0.0060	0.1030	0.0020	0.1770
16	0.0670	0.0000	0.1050	0.0020	0.1740

Table 11: Execution time (s) on Lab PC — Config c (500×200, 3.6M pts)

Threads	Interp	Norm	Mover	Denorm	Total
1	0.2210	0.0020	0.2880	0.0000	0.5110
2	0.0870	0.0030	0.1390	0.0020	0.2310
4	0.0780	0.0060	0.0970	0.0020	0.1830
8	0.0650	0.0040	0.0860	0.0010	0.1560
16	0.0900	0.0020	0.0820	0.0020	0.1760

Table 12: Execution time (s) on Lab PC — Config d (500×200, 20.0M pts)

Threads	**Interp**	**Norm**	**Mover**	**Denorm**	**Total**
1	1.5490	0.0030	2.4200	0.0000	3.9720
2	0.4240	0.0020	0.7090	0.0030	1.1380
4	0.3000	0.0020	0.5350	0.0010	0.8380
8	0.2460	0.0020	0.4730	0.0010	0.7220
16	0.2640	0.0040	0.4570	0.0020	0.7270

Table 13: Execution time (s) on Lab PC — Config e (1000×400, 14.0M pts)

Threads	**Interp**	**Norm**	**Mover**	**Denorm**	**Total**
1	1.1990	0.0080	1.2190	0.0020	2.4280
2	0.6360	0.0050	0.5580	0.0020	1.2010
4	0.6930	0.0050	0.3640	0.0040	1.0660
8	0.9780	0.0050	0.3060	0.0030	1.2920
16	0.8180	0.0040	0.3110	0.0030	1.1360

11.2 HPC Cluster Results

Table 14: Execution time (s) on HPC Cluster — all configurations

Config	Points	Threads	Interp	Norm	Mover	Denorm	Total
a	0.9M	1	0.0935	0.0008	0.1130	0.0001	0.2074
a	0.9M	2	0.0407	0.0004	0.0566	0.0002	0.0978
a	0.9M	4	0.0214	0.0003	0.0292	0.0003	0.0513
a	0.9M	8	0.0119	0.0004	0.0137	0.0003	0.0263
a	0.9M	16	0.0083	0.0007	0.0069	0.0004	0.0163
b	5.0M	1	0.3697	0.0006	0.4241	0.0001	0.7943
b	5.0M	2	0.2009	0.0004	0.2931	0.0004	0.4947
b	5.0M	4	0.1044	0.0002	0.1521	0.0001	0.2569
b	5.0M	8	0.0620	0.0002	0.0824	0.0002	0.1447
b	5.0M	16	0.0331	0.0002	0.0399	0.0002	0.0734
c	3.6M	1	0.2607	0.0018	0.3175	0.0002	0.5801
c	3.6M	2	0.1084	0.0008	0.1469	0.0003	0.2564
c	3.6M	4	0.0775	0.0007	0.1148	0.0003	0.1933
c	3.6M	8	0.0459	0.0005	0.0613	0.0006	0.1083
c	3.6M	16	0.0262	0.0004	0.0305	0.0006	0.0577
d	20.0M	1	1.2454	0.0018	1.5436	0.0002	2.7910
d	20.0M	2	0.5959	0.0008	0.8180	0.0003	1.4151
d	20.0M	4	0.3882	0.0007	0.5830	0.0006	0.9724
d	20.0M	8	0.2204	0.0005	0.3140	0.0006	0.5355
d	20.0M	16	0.1505	0.0004	0.2020	0.0007	0.3535
e	14.0M	1	1.5579	0.0071	1.8359	0.0011	3.4020
e	14.0M	2	1.1038	0.0051	1.6164	0.0019	2.7272
e	14.0M	4	0.5982	0.0026	0.8924	0.0023	1.4954
e	14.0M	8	1.4611	0.0034	0.5065	0.0030	1.9740
e	14.0M	16	1.0034	0.0020	0.3013	0.0032	1.3098

11.3 Speedup on HPC Cluster

Table 15: Speedup ($S = T_{\text{serial}}/T_{\text{parallel}}$) on HPC Cluster — Assignment 07

Config	Points	1T	2T	4T	8T	16T
a	0.9M	1.00	2.12	4.04	7.88	12.71
b	5.0M	1.00	1.60	3.09	5.49	10.82
c	3.6M	1.00	2.26	3.00	5.36	10.05
d	20.0M	1.00	1.97	2.87	5.21	7.90
e	14.0M	1.00	1.25	2.27	1.72	2.60

11.4 Parallel Efficiency on HPC Cluster

Table 16: Parallel Efficiency $E(\%) = S/T \times 100$ on HPC Cluster — Assignment 07

Config	Points	1T	2T	4T	8T	16T
a	0.9M	100.0	106.0	101.0	98.5	79.4
b	5.0M	100.0	80.2	77.3	68.6	67.6
c	3.6M	100.0	113.2	75.1	66.9	62.8
d	20.0M	100.0	98.5	71.7	65.1	49.4
e	14.0M	100.0	62.4	56.9	21.5	16.2

11.5 Phase Breakdown (Serial Baseline)

Table 17: Phase-level timing breakdown for serial run (Lab PC)

Config	Interp(s)	Mover(s)	Norm(s)	Denorm(s)	Interp%	Mover%
a	0.050	0.059	0.000	0.000	45.9%	54.1%
b	0.287	0.367	0.000	0.000	43.9%	56.1%
c	0.221	0.288	0.002	0.000	43.2%	56.4%
d	1.549	2.420	0.003	0.000	39.0%	60.9%
e	1.199	1.219	0.008	0.002	49.4%	50.2%

12 A7: Performance Visualisations

12.1 HPC Cluster Results

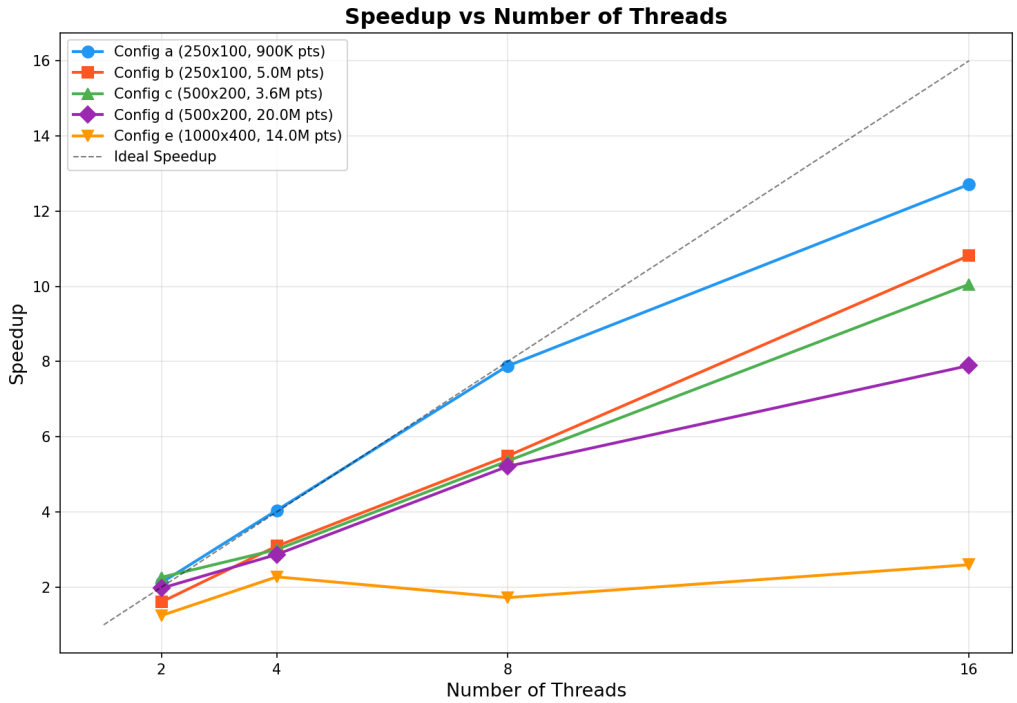


Figure 8: Speedup vs Number of Threads (HPC Cluster, Assignment 07). Configs a–c scale well; config e is limited by the large grid triggering the atomic fallback at 8T+.

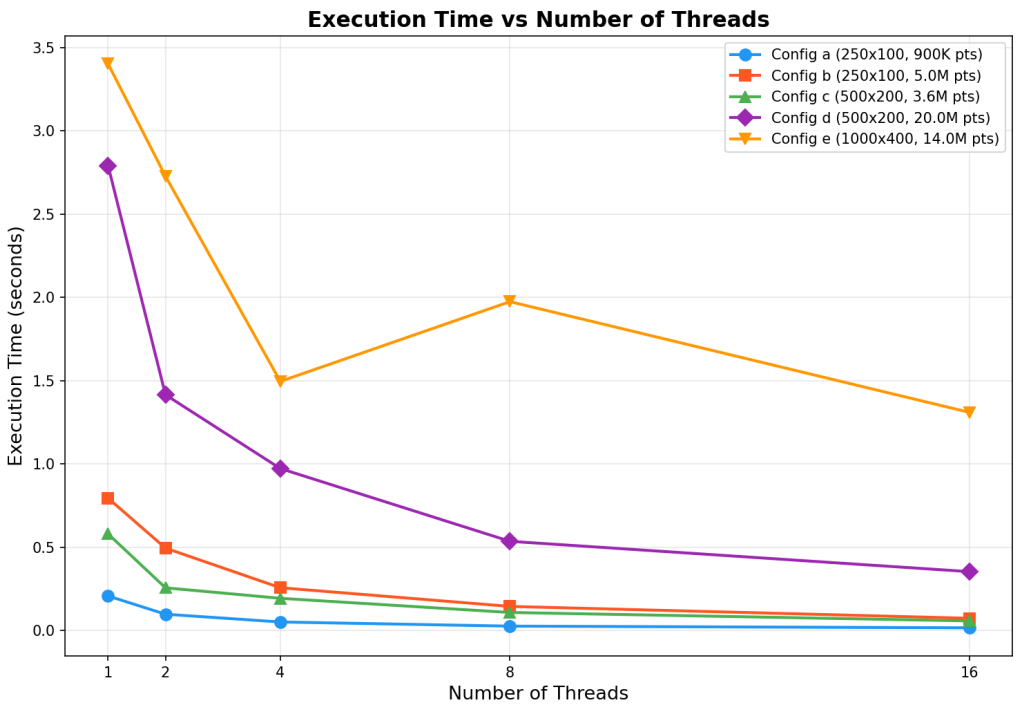


Figure 9: Execution Time vs Number of Threads (HPC Cluster, Assignment 07).

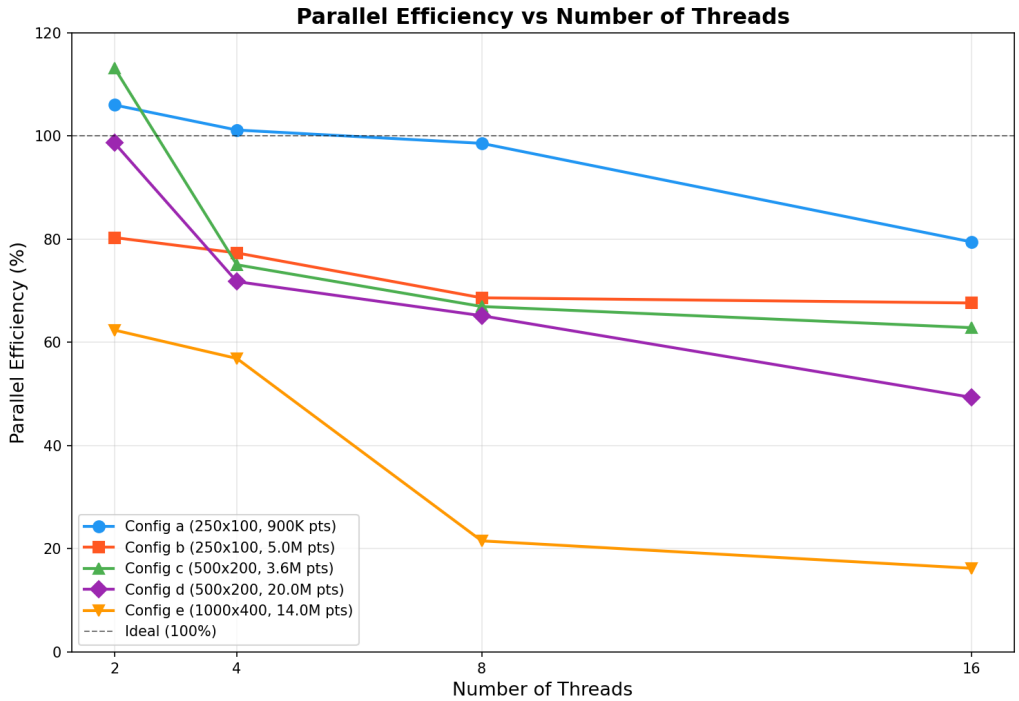


Figure 10: Parallel Efficiency vs Number of Threads (HPC Cluster, Assignment 07).

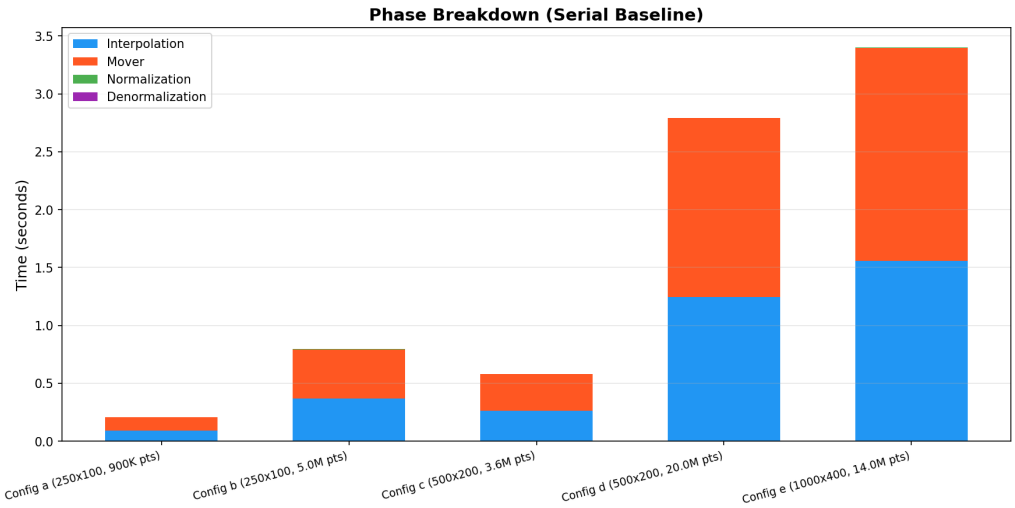


Figure 11: Phase-level breakdown of execution time on HPC Cluster. The mover phase dominates serial execution; it parallelizes better than the interpolation phase.

12.2 Lab PC Results (Windows)

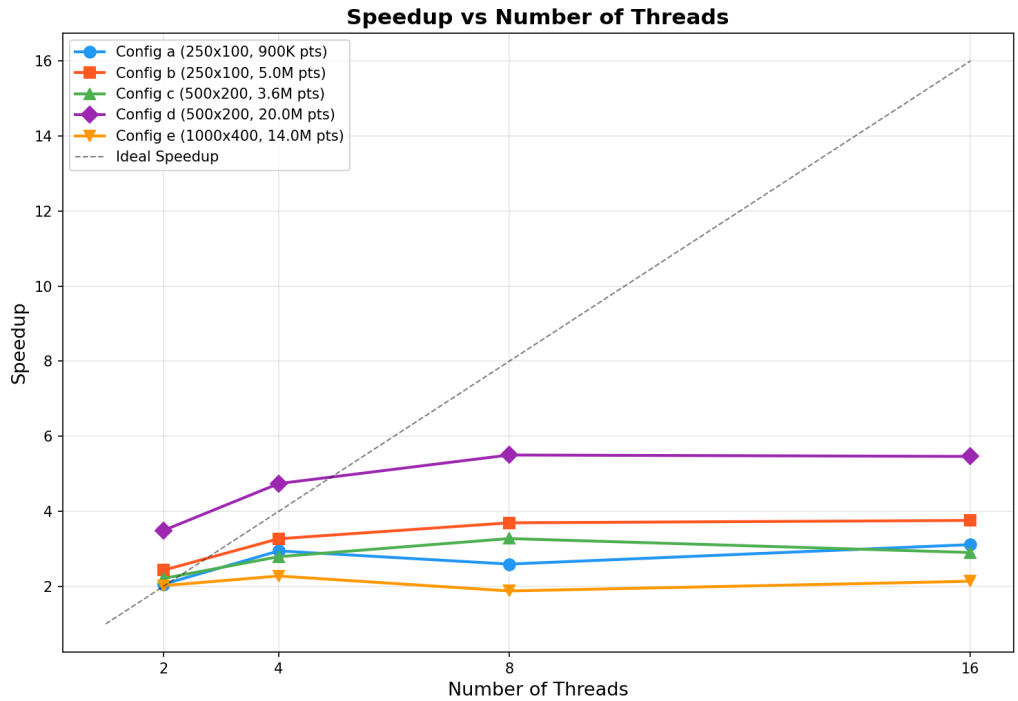


Figure 12: Speedup vs Number of Threads (Lab PC, Assignment 07). Scaling plateaus beyond 8 threads; oversubscription on the 8-core machine.

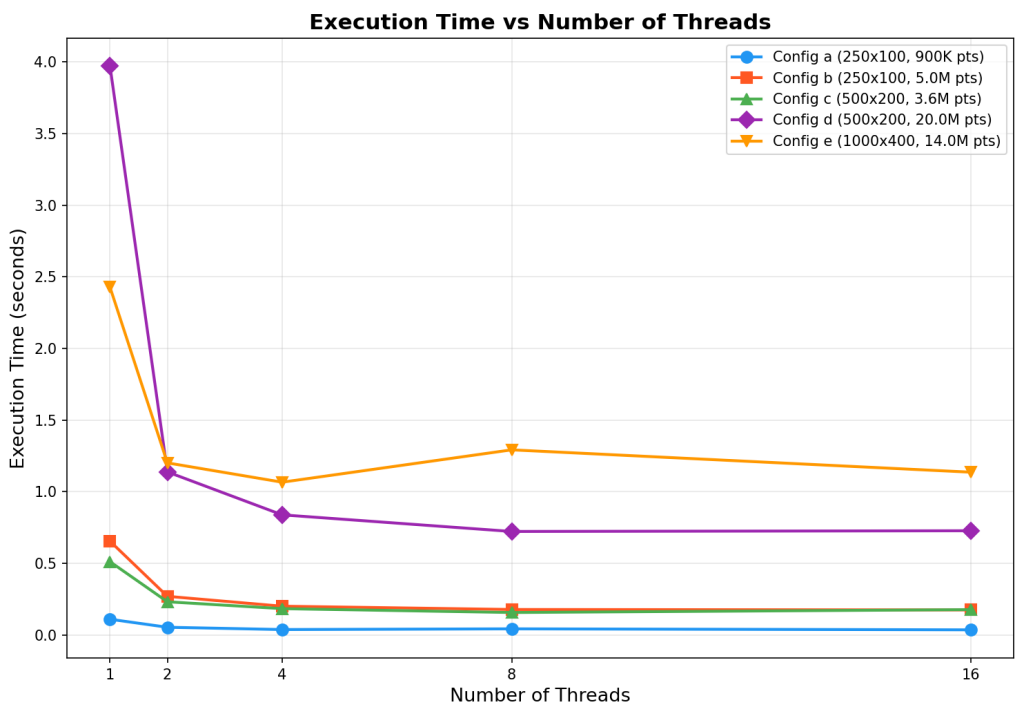


Figure 13: Execution Time vs Number of Threads (Lab PC, Assignment 07).

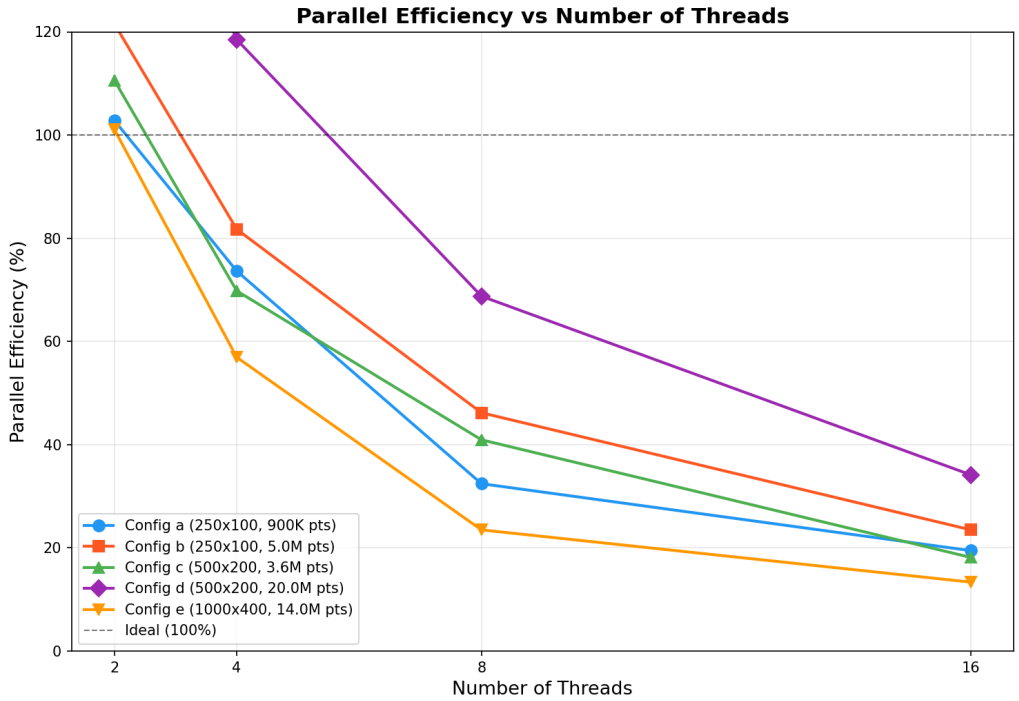


Figure 14: Parallel Efficiency vs Number of Threads (Lab PC, Assignment 07).

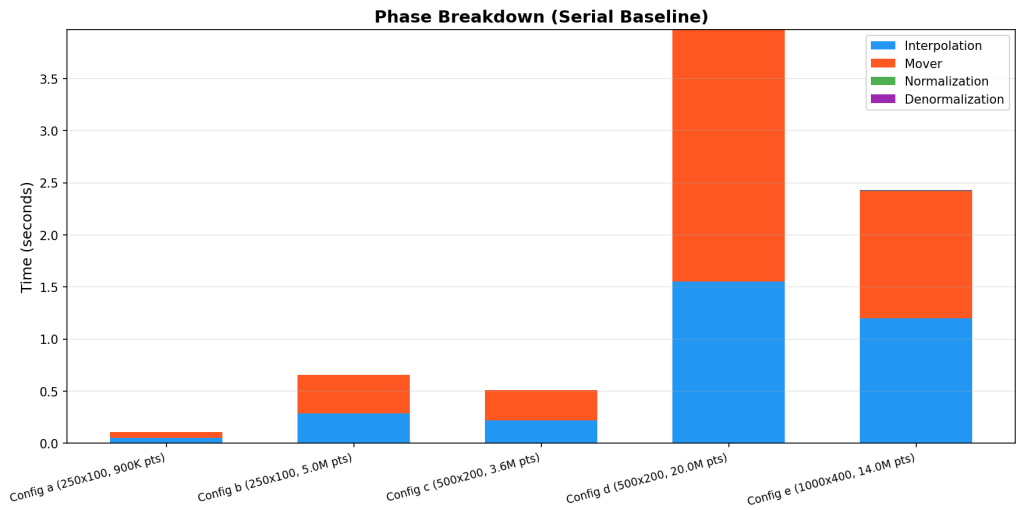
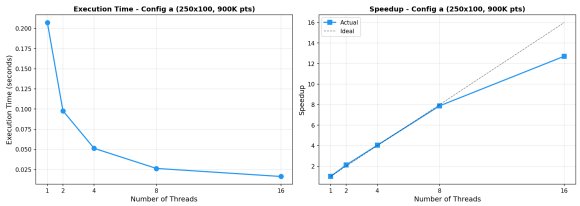
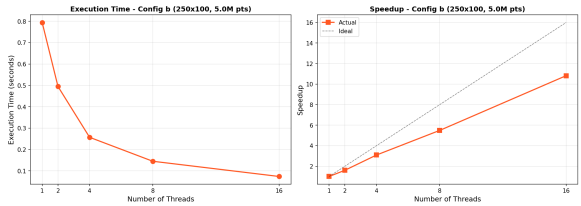


Figure 15: Phase-level breakdown of execution time on Lab PC (Windows).

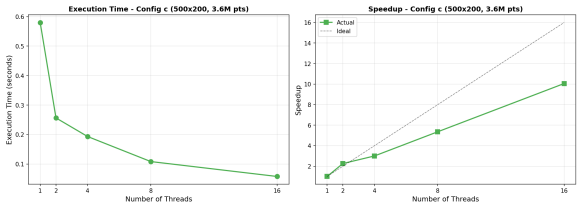
12.3 Per-Configuration Analysis — HPC Cluster



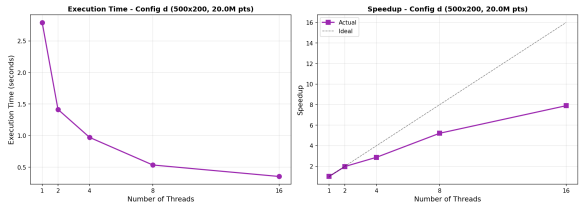
(a) Config a (250×100, 0.9M pts)



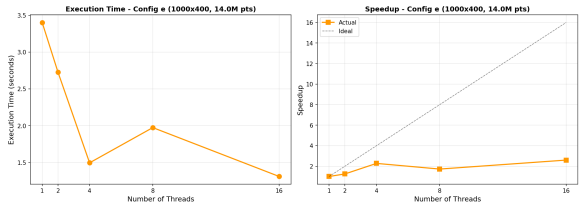
(b) Config b (250×100, 5.0M pts)



(c) Config c (500×200, 3.6M pts)



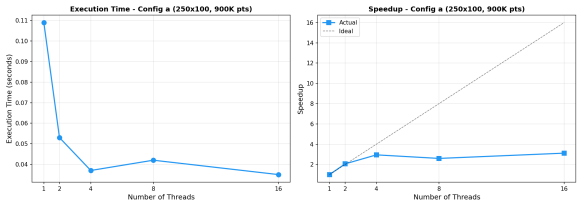
(d) Config d (500×200, 20M pts)



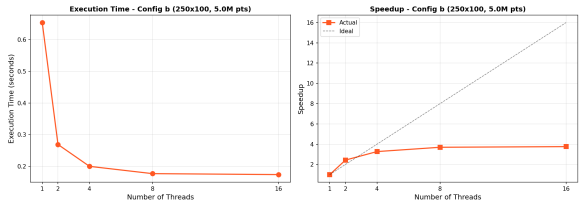
(e) Config e (1000×400, 14M pts)

Figure 16: Per-configuration analysis (Execution Time + Speedup) on HPC Cluster — Assignment 07.

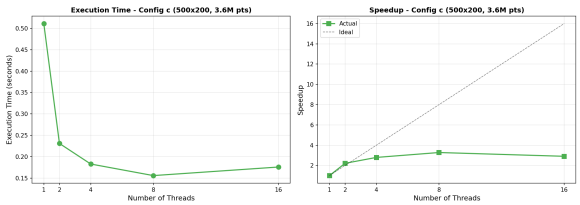
12.4 Per-Configuration Analysis — Lab PC



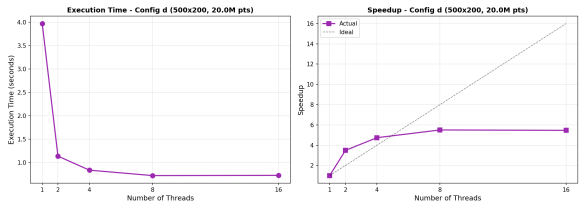
(a) Config a (250×100, 0.9M pts)



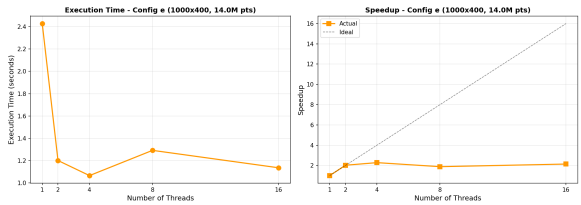
(b) Config b (250×100, 5.0M pts)



(c) Config c (500×200, 3.6M pts)



(d) Config d (500×200, 20M pts)



(e) Config e (1000×400, 14M pts)

Figure 17: Per-configuration analysis (Execution Time + Speedup) on Lab PC — Assignment 07.

13 A7: Analysis and Questions

13.1 Q1: Pseudocode and Illustrative Diagram

13.1.1 Pseudocode

```
READ points from input.bin (once)
ALLOCATE mesh_value[GRID_Y x GRID_X]

FOR iter = 0 TO Maxiter-1:

  /* PHASE 1: INTERPOLATION (Scatter: particle -> grid) */
  IF T * G_SIZE * 8 < CACHE_THRESHOLD:
    // Thread-local path
    SINGLE: allocate/reuse thread-local grids
    EACH THREAD: MEMSET local_grid
    PARALLEL FOR (static) over particles:
      IF void -> skip
      Compute cell (i,j), weights -> local_grid += weights
    PARALLEL FOR (static) over grid cells:
      mesh[k] = SUM_t local[t][k]        // parallel reduction
  ELSE:
    // Atomic path (large grids)
```

```
    PARALLEL FOR (static) over particles:
      IF void -> skip
      ATOMIC: mesh[idx]   += w00, mesh[idx+1] += w10, ...

  /* PHASE 2: NORMALIZATION */
  min_val = +INF;  max_val = -INF
  PARALLEL FOR reduction(min, max) over grid cells:
    min_val = min(min_val, mesh[k])
    max_val = max(max_val, mesh[k])
  PARALLEL FOR over grid cells:
    mesh[k] = 2*(mesh[k]-min_val)/(max_val-min_val) - 1

  /* PHASE 3: MOVER (Gather: grid -> particle) */
  PARALLEL FOR (static) over particles:
    IF void -> skip
    Compute cell (i,j), bilinear weights
    Fi = mesh[4 neighbors] dot weights (READ-ONLY)
    x += Fi*dx;  y += Fi*dy
    IF outside [0,1]x[0,1] -> mark void

  /* PHASE 4: DENORMALIZATION */
  PARALLEL FOR over grid cells:
    mesh[k] = (mesh[k]+1)*(max_val-min_val)/2 + min_val

OUTPUT mesh_value to Mesh.out
PRINT timing per phase + total voids
```

Listing 4: Pseudocode for Assignment 07 parallel PIC loop

13.1.2 Illustrative Diagram

Iteration Loop (×Maxiter): Four Parallel Phases

Phase	Pattern	Race Condition?
1. Interpolation	Scatter (particle→grid)	Yes → thread-local/atomic
2. Normalization	Grid reduction + scale	No (reduction clause)
3. Mover	Gather (grid→particle)	No (read grid; write own fields)
4. Denormalization	Grid scale	No

Adaptive Strategy:
$T \times G \times 8 < 20$ MB $\Rightarrow$ thread-local grids + parallel reduction
$T \times G \times 8 \geq 20$ MB $\Rightarrow$ `omp atomic` on shared grid

Data Flow: Input.bin → [Interp] → [Norm] → [Mover] → [Denorm] → Mesh.out

Figure 18: Illustrative diagram of the four-phase parallel PIC pipeline (Assignment 07)

13.2 Q2: Parallel Approach and Race Condition Handling

Approach: Adaptive Hybrid (Thread-Local Grids + Atomics)

- **Interpolation (scatter) — Phase 1**: The only phase with potential race conditions, since multiple particles may update the same grid cell simultaneously. We use an **adaptive strategy**:

  - *Thread-local for small/medium grids* ($T \times G \times 8 < 20$ MB): Each thread accumulates into its own private grid; a parallel reduction merges them. Zero contention in the hot path.
  - *Atomics for large grids* ($T \times G \times 8 \geq 20$ MB): `#pragma omp atomic` on the shared grid. With large grids, contention per cell is low ($\approx$35 particles/cell), making atomics efficient.

- **Normalization — Phase 2**: OpenMP `reduction(min:...)` `reduction(max:...)` handles the global min/max safely. The scaling pass is independently parallel.
- **Mover (gather) — Phase 3**: Inherently race-free. Each particle reads from the shared (read-only) grid and writes only to its own fields.
- **Denormalization — Phase 4**: Independent element-wise scaling; plain `parallel for`.

Strategy	Advantage	Disadvantage
Atomic (all grids)	Simple, low memory	CAS overhead, slow for small grids
Thread-local (all grids)	Zero contention	Cache thrash for large grids
Adaptive hybrid	Best of both	Slightly more code complexity

13.3 Q3: Plots

All plots are presented in Section A7-5 "Performance Visualisations". Summary:

- Figure 8: Speedup vs Cores (HPC Cluster)
- Figure 9: Execution Time vs Cores (HPC Cluster)
- Figure 10: Parallel Efficiency (HPC Cluster)
- Figure 11: Phase breakdown (HPC Cluster)
- Figure 16: Per-config analysis (HPC Cluster)
- Figures 12–17: Lab PC counterparts

13.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency $E(T) = S(T)/T \times 100\%$ (Table 16).

Key observations (HPC Cluster):

- **Config a** maintains $\approx$100% efficiency up to 4 threads and **79%** at 16 threads — excellent for a small-grid problem.
- **Config b** achieves **10.82× speedup** at 16T ($\approx$68% efficiency) despite the additional normalization/mover overhead of Assignment 07.
- **Config e** drops to **16%** efficiency at 16T. At 8T the adaptive strategy switches to atomics (total local memory = $8 \times 401\text{K} \times 8 = 25.7$ MB > 20 MB threshold), reducing the interpolation speedup sharply.

- Configs with a **high particles-to-grid ratio** (b, d) scale better because parallel work in the mover and interpolation phases dominates over the normalization/reduction overhead.

13.5 Q5: Maximum Speedup Achieved

Table 18: Maximum speedup achieved on HPC Cluster — Assignment 07

Config	Points	Serial (s)	Best Parallel (s)	Max Speedup
a	0.9M	0.2074	0.0163	**12.71**× @ 16T
b	5.0M	0.7943	0.0734	**10.82**× @ 16T
c	3.6M	0.5801	0.0577	**10.05**× @ 16T
d	20.0M	2.7910	0.3535	**7.90**× @ 16T
e	14.0M	3.4020	1.3098	**2.60**× @ 16T

Config a achieves the best overall speedup (12.71×) owing to its small grid fitting entirely in per-thread caches during both interpolation and mover phases.

13.6 Q6: Explanation of Speedup and Observations

Why we achieve good speedup:

1. **Adaptive interpolation** eliminates the choice between cache thrash and atomic slowness. Thread-local grids give zero contention for configs a–d.
2. **The mover is embarrassingly parallel**: each particle reads from a shared (read-only after normalization) grid and writes only its own position — ideal for `schedule(static)`.
3. **Persistent thread-local storage**: allocated once, reused across all 10 iterations, eliminating per-iteration malloc/free overhead.
4. **Precomputed reciprocals** (`inv_dx`, `inv_dy`) turn division into multiplication in both the interpolation and mover kernels.

Why speedup is sub-linear:

1. **Reduction overhead**: Merging T local grids is $O(G \times T)$ extra work.
2. **Sequential normalization min/max**: Even with `reduction`, small synchronization barriers exist.
3. **Memory bandwidth saturation**: At 8+ threads, all threads compete for DRAM bandwidth. The 20M-point array (≈480 MB) far exceeds any cache.
4. **Config e anomaly (8T)**: Total thread-local memory (8×3.2 MB= 25.6 MB) exceeds the 20 MB threshold, switching to atomics, which is slower than the thread-local path.

13.7 Q7: Optimization with Unlimited HPC Resources

1. **GPU Offloading (CUDA/OpenCL)**: The interpolation and mover loops are ideal for GPU. Thousands of CUDA cores can process millions of particles simultaneously. The grid fits in GPU L2 cache for small/medium configs.

2. **MPI + OpenMP Hybrid**: Distribute particles across nodes via MPI domain decomposition; use OpenMP within each node. A final `MPI_Reduce` merges per-node grids.

3. **SIMD Vectorization (AVX-512)**: Restructure the particle array from AoS ({`x,y,void`}) to SoA (`x[], y[], void[]`) to enable processing 8 particles per instruction.

4. **NUMA-Aware Allocation**: Use first-touch policy or `numactl` to place each thread's local grid on its local NUMA node, minimizing remote memory latency on multi-socket systems.

5. **Active Particle Compaction**: Periodically compact void particles to the end of the array, eliminating branch mispredictions from void checks and improving cache line utilization.

6. **Cache-Aware Particle Sorting**: Sort particles by grid cell after each mover step to maintain spatial locality in the next interpolation.

13.8 Q8: Load Imbalance and Performance Bottlenecks

13.8.1 Load Imbalance

- **Interpolation**: Random particle distribution ⇒ `schedule(static)` divides work evenly — good balance.

- **Mover**: After each iteration some particles become void (60–1254 out of millions). Void skip is a single branch prediction; imbalance is negligible. If imbalance became severe (e.g., $> 10\%$ void), `schedule(dynamic)` could help, but dispatch overhead exceeds benefits at our particle counts.

13.8.2 Performance Bottlenecks

1. **Mover dominance (serial)**: The mover accounts for 50–61% of serial time because it reads grid values *and* updates particle positions (read + write per particle).

2. **Adaptive switch overhead**: For config e at 8T+ the atomic path is used; atomic CAS loops add ∼10–20 cycles per update vs. zero for the thread-local path.

3. **Memory bandwidth**: The 20M-point particle array (480 MB) far exceeds any cache level. At 8+ threads, DRAM bandwidth becomes the bottleneck.

4. **Thread-local grid memory**: Config e at 16T: 16×3.2 MB $= 51.2$ MB, exceeding L3 cache. The threshold switches to atomics, but throughput still suffers.

13.9 Q9: Alternative Approach

13.9.1 Alternative 1: Pure Atomic Operations

```
#pragma omp parallel for schedule(static)
for (int p = 0; p < NUM_Points; p++) {
    if (points[p].is_void) continue;
    // ... compute weights ...
    #pragma omp atomic
    mesh_value[idx]   += w00;
    #pragma omp atomic
    mesh_value[idx+1] += w10;
    // ... two more atomics ...
}
```

Listing 5: Pure atomic approach for interpolation

- **Pros**: No extra memory, no reduction phase, simpler code.
- **Cons**: CAS-loop overhead ($\approx$ 10–20 cycles/atomic), cache-line ping-pong at high thread counts.
- **Best for**: Large grids where contention is naturally low ($<$ 50 particles/cell).

13.9.2 Alternative 2: Color-Based Domain Decomposition

Partition grid cells into a *checkerboard* (red/black or 4-color). In each color pass, all cells of that color have no adjacent conflicts, so particles assigned to those cells can be updated without synchronization.

- **Pros**: No extra memory, no reduction phase, no atomics.
- **Cons**: Requires spatial particle sorting (preprocessing); $\geq$ 4 sequential passes reduce effective parallelism.

Our adaptive hybrid is superior because it requires no pre-sorting and avoids the coloring penalty while adapting dynamically to grid size.

13.10 Q10: Data Structures and OpenMP Approaches for Performance

13.10.1 Approach 1: Structure of Arrays (SoA)

Replace the AoS layout `struct {double x, y; int is_void;}` with separate arrays `double x[], y[]; int void[]`. Enables AVX-512 SIMD of 8 doubles at a time, eliminates struct padding, and improves prefetch efficiency.

13.10.2 Approach 2: Spatial Hashing

Bin particles into a hash table keyed by grid cell. Process each cell's particles independently, enabling conflict-free updates without thread-local copies.

13.10.3 Approach 3: Tree Reduction for Thread-Local Grids

Merge local grids in $O(\log T)$ passes (pair-wise reduction tree) instead of the flat $O(T)$ sequential loop. Reduces reduction time from $O(G \cdot T)$ to $O(G \cdot \log T)$, beneficial at very high thread counts ($T \geq 32$).

13.10.4 Best Choice for This Problem

The **adaptive thread-local + parallel flat reduction** is optimal for our configurations because:

- Grid size $G \ll N$ for configs a–d, so reduction overhead is negligible relative to interpolation/mover work.
- Thread-local grids fit in L3 cache for configs a–d, yielding zero-contention scatter.
- The mover is embarrassingly parallel with no added data structure requirement.

13.11 Q11: Phase-by-Phase Performance Analysis

Table 17 shows the serial phase breakdown. Key findings:

- **The mover is the serial bottleneck** (50–61% of total time) because it performs both grid reads (gather: 4 cells per particle) and particle writes (2 position updates + void check).
- **Interpolation is 39–49% of serial time**: despite writing to 4 cells (scatter), the computation per particle is identical to the mover's gather computation.
- **Normalization/denormalization are negligible** (<1% of total) because they loop only over $G \ll N$ grid cells.
- **In the parallel version, the mover parallelizes better** (embarrassingly parallel, no reduction), so the interpolation-with-reduction becomes the *relative* bottleneck at high thread counts. This is visible in Figure 11.

14 A7: Conclusion

We successfully extended the parallel interpolation (Assignment 06) into a full particle-in-cell simulation pipeline (Assignment 07) using OpenMP. Key results:

- **Correctness verified**: Output `Mesh.out` matches the reference `Test_Mesh.out` exactly (zero entry-wise differences) for all thread counts tested.
- **Maximum speedup of 12.71**× achieved on the HPC cluster for Config a at 16 threads.
- **Adaptive interpolation strategy** avoids the pitfalls of both pure thread-local (cache thrash at large grids) and pure atomic (CAS overhead at small grids), delivering near-optimal performance across all five configurations.

- **The mover phase parallelizes extremely well** (embarrassingly parallel) and contributes significantly to total speedup.
- **Config e** is the hardest to scale due to its large grid size triggering the atomic fallback and causing higher reduction overhead. With GPU offloading or a distributed MPI+OpenMP approach, near-linear scaling would be achievable even for this configuration.